arXiv:2404.07129v1 [cs.LG] 10 Apr 2024

# What needs to go right for an induction head? A mechanistic study of in-context learning circuits and their formation

**Aaditya K. Singh** [1] **Ted Moskovitz** [1] **Felix Hill** [2] **Stephanie C. Y. Chan** [*2] **Andrew M. Saxe** [*1]

## Abstract

In-context learning is a powerful emergent ability in transformer models. Prior work in mechanistic interpretability has identified a circuit element that may be critical for in-context learning – the induction head (IH), which performs a match-and-copy operation. During training of large transformers on natural language data, IHs emerge around the same time as a notable phase change in the loss. Despite the robust evidence for IHs and this interesting coincidence with the phase change, relatively little is known about the diversity and emergence dynamics of IHs. Why is there more than one IH, and how are they dependent on each other? Why do IHs appear all of a sudden, and what are the subcircuits that enable them to emerge? We answer these questions by studying IH emergence dynamics in a controlled setting by training on synthetic data. In doing so, we develop and share a novel optogenetics-inspired causal framework for modifying activations throughout training. Using this framework, we delineate the diverse and additive nature of IHs. By *clamping* subsets of activations throughout training, we then identify three underlying subcircuits that interact to drive IH formation, yielding the phase change. Furthermore, these subcircuits shed light on data-dependent properties of formation, such as phase change timing, already showing the promise of this more in-depth understanding of subcircuits that need to "go right" for an induction head.

## 1. Introduction

Large language models (LLMs) trained on internet scale corpora (Brown et al., 2020) showcase a remarkable ability to perform *in-context learning* (ICL), adapting to new inputs and tasks at test time. Given the increasing prevalence of LLMs, safety researchers have sought to understand the mechanisms underlying important phenomena like ICL. One approach in the service of this goal is *mechanistic interpretability*: reverse engineering the computations performed by a model. Olsson et al. (2022) identified the induction circuit, which may be responsible for many of transformers' ICL abilities.

An induction circuit is a two-layer circuit (Figure 1a) comprised of a "previous token head" in an earlier layer, followed by the eponymous "induction head". Previous token heads are attention heads responsible for attending to the previous token and copying it into the attending token's residual stream.[1] Induction heads then perform a match-and-copy operation, looking for a match between a query derived from the current token and key derived from the output of the previous token head. Critically, induction circuits typically emerge in a sharp phase change in the loss (e.g., Figure 3), which is often believed to be due to the interaction between two heads in different layers (Nanda, 2022). Presumably, neither the previous token nor the induction head are useful on their own for minimizing the loss.

Recent empirical work on ICL in transformers has introduced more nuances (Min et al., 2022; Wei et al., 2023b), with Singh et al. (2023) even finding that ICL can be a transient phenomenon that disappears with overtraining. These results establish a need to more thoroughly understand induction circuits, especially the *dynamics* of formation during training. Reddy (2023) begins to address this question using *progress measures*, which track correlational relationships between intermediate activations during training. We instead use a causal approach, termed *clamping*, which allows us to directly determine the circuits and dynamics that are causally affecting formation. Through these experiments, we bring to light a new set of mechanisms governing learning dynamics.

In this work, we take inspiration from the field of optogenetics in neuroscience, which allows precise causal manipu-

[*]Equal contribution [1]Gatsby Computational Neuroscience Unit, University College London [2]Google DeepMind. Correspondence to: Aaditya K. Singh <aaditya.singh.21@ucl.ac.uk>.

[1]The residual stream was a term introduced by Elhage et al. (2021) and refers to the embeddings after each layer that further layers "read" input from. Layers can be viewed as reading from and writing back into this stream. The term "stream" is meant to emphasize the residual skip connections.

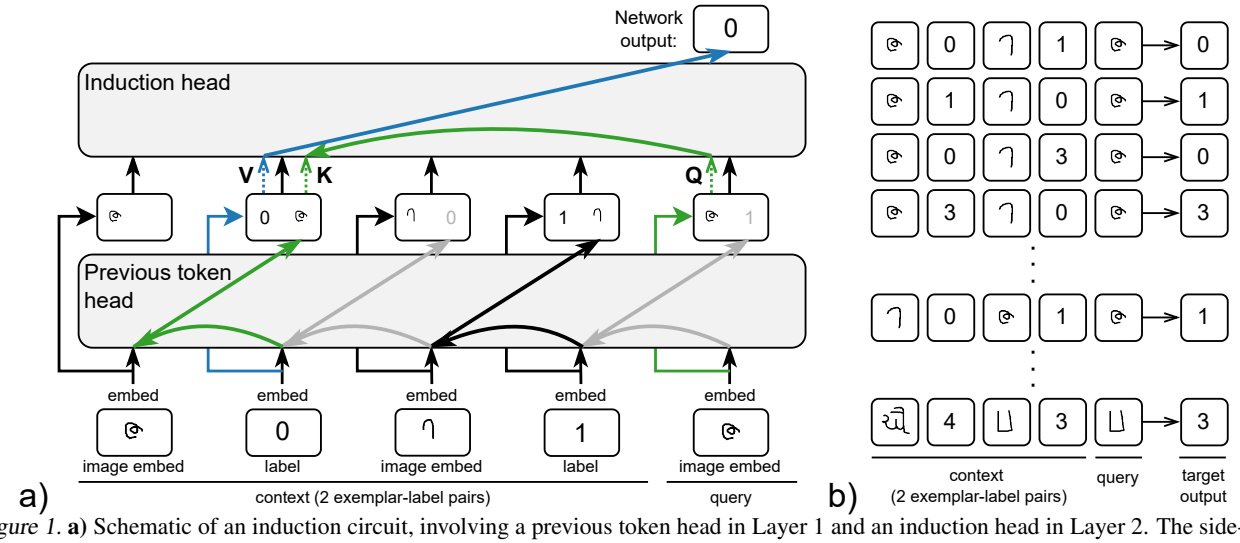


*Figure 1.* **a)** Schematic of an induction circuit, involving a previous token head in Layer 1 and an induction head in Layer 2. The side-by-side labels and exemplars in the residual stream after Layer 1 are meant to indicate that information about both is superimposed (perhaps in different subspaces). We highlight[3]the "matching" (green) and "copying" (blue) operations that span the two layers. Historically, focus has been devoted to the "match" operation. One of our key results is to demonstrate the important interactions from the "copy" operation. **b)** Example training sequences built from the Omniglot dataset and inspired by classical few-shot meta-training. The context consists of two exemplar-label pairs, where the exemplars are from different classes. The query exemplar comes from the same class as one of the exemplars in context. The in-context labels are randomly chosen. Every exemplar can appear with every possible label in every possible position, forcing the transformer to use ICL to minimize the training loss. Validation sequences either use held out class exemplars or held out pairs of labels.

lations of neural activity (Boyden et al., 2005). Here, we develop a novel framework for causally modifying activations throughout model training. These causal-through-training clamping analyses extend beyond prior work, allowing us to easily study interactions between subcircuits and isolate the underlying factors that drive induction circuit formation. We utilize this framework to analyze induction circuit formation in transformers trained to solve a simple few-shot learning (FSL) task inspired by prior work (Chan et al., 2022; Lake et al., 2015). As with any work towards a mechanistic understanding of dynamics, we first provide a thorough investigation of the induction circuits in a network trained on this task, shedding light on the dependence (and independence) of induction heads upon each other (Section 3). We find that multiple induction heads form, contributing additively to minimize the loss. Furthermore, we observe emergent redundancy, despite not applying regularization techniques such as dropout (Hinton et al., 2012), mirroring findings on larger-scale language models (Michel et al., 2019; Voita et al., 2019). We also find the wiring between induction heads and previous token heads to be many-to-many, rather than one-to-one. Next, we focus on the dynamics of formation (Section 4), where we use clamping (causal manipulations of activations throughout training) to identify three, smoothly evolving underlying subcircuits whose interaction may be causing the seemingly discontinuous phase change. Specifically, the third key evolving subcircuit is responsible for copying input labels to the output (highlighted in blue, Figure 1a), a function that was often believed to be easy (with most prior work focusing on the "match" operation instead, where an induction circuit has to find the right token to attend to). Finally, we show how data properties influence the timing of the phase change, and how this shift in induction circuit formation can be better understood by individually understanding the data-dependent formation of each of our identified subcircuits.

As a resource to the community, we open-source our codebase at `https://github.com/aadityasingh/icl-dynamics`, providing a tool for modifying activations throughout training and conducting further causal analyses of transformers' circuit elements. Our work and tooling represents an important step in mechanistic understanding of training dynamics, applied here to induction circuit formation, and we hope it spurs further progress on understanding how different computations in transformers are learned.

## 2. Methods

### 2.1. Experimental setup

We train transformer models on a FSL task. Sequences are series of exemplar-label pairs, followed by a query exemplar (see Figure 1b). Exemplars come from the Omniglot dataset (Lake et al., 2015), a common benchmark for FSL also used by prior work on ICL in transformers (Chan et al., 2022;

[3]We also gray out the previous token head operation for copying label tokens. This behavior does not robustly emerge as it isn't necessary since we only train on sequences of a fixed length.

```
def example_pattern_preserving_ablation(model, sequence, preserve, ablate):
  # preserve - list of (layer, head) pairs to preserve patterns on
  # ablate - list of (layer, head) pairs to ablate
  base_cache = model.call_with_all_aux(sequence, cache=None, cache_mask=None)
  cache = jax.tree_map(jnp.zeros_like, base_cache)
  cache_mask = jax.tree_map(partial(jnp.zeros_like, dtype=bool), base_cache)
  # preserve head patterns
  for layer, head in preserve:
    cache[layer][head]['attn_pattern'] = base_cache[layer][head]['attn_pattern']
    cache_mask[layer][head]['attn_pattern'] = True
  # perform other ablation, example: turning off heads
  for layer, head in ablate:
    cache[layer][head]['out'] = 0
    cache_mask[layer][head]['out'] = True
  return model(sequence, cache=cache, cache_mask=cache_mask)
```

*Figure 2.* Example pseudocode demonstrating a pattern preserving ablation using our framework.

Singh et al., 2023). Omniglot consists of 1600 classes of 20 exemplars. Most of our experiments use a simplified dataset with a random set of $C = 50$ classes and $E = 1$ exemplar per class, which we found sufficient to elicit the relevant phenomena. To obtain input embeddings, we feed Omniglot images through an ImageNet-pretrained, frozen Resnet18 encoder (He et al., 2015; Russakovsky et al., 2015).

To isolate FSL capabilities, we use the standard meta-training setup where labels are randomized for each sequence. Specifically, each sequence can be viewed as a 2-way, 1-shot classification problem (see Figure 1b). Labels are randomly selected from $L$ one-hot labels. To obtain input embeddings, we use a standard learnt embedding layer.

In our default setting[4] ($C = 50, E = 1, L = 5$), we have a total of 78400 unique training sequences (Appendix C). On this data, we train causal, 2-layer attention-only transformer models, as used by prior work (Olsson et al., 2022), to study induction circuit formation. Model and optimizer hyperparameters can be found in Appendix A.

To test generalization, we consider two test sets over held-out classes and relabelings. Specifically, for "test (exemplars)", we evaluate performance on a random, held-out set of $C_{test} = 100$ classes, which verifies the generality of the "match" operation of induction circuits. For "test (relabel)", we hold out and test on a fixed percentage (20%) of pairs of labels. This ensures that, while all labels are seen during training, not all pairs are. This test set verifies the generality of the "copy" operation of induction circuits.

### 2.2. An artificial-optogenetics framework

One of the contributions of this work is a novel training and analysis framework that easily exposes activations, and allows causal manipulations throughout training (as opposed to only after training, as in prior work). The framework enables a wide range of analyses—in this work, we use it to conduct targeted analyses of induction circuit formation via clamping. Our codebase is implemented in Equinox/JAX (Kidger & Garcia, 2021; Bradbury et al., 2018), natively supports up to 50x speed-ups with `jax.jit`,[5] and is open-source.

In contrast to the standard practice of having modules with a forward function implemented as `__call__`, we have an underlying `call_with_all_aux` method which returns a pytree of all intermediate activations. The `__call__` method wraps `call_with_all_aux` and returns just the output. During the training process, `__call__` is used. For analysis, `call_with_all_aux` can be easily used to expose all intermediate activations.

To allow for editing/ablating activations throughout training (or just post-hoc, as done in prior work), `call_with_all_aux` accepts `cache` and `cache_mask` arguments, which have the same shape as the output pytree of `call_with_all_aux`. These caches can be used to insert activations into the network, e.g. clamping activations at particular values during training. Combined with the functional, automatic differentiation of JAX, these caches can be input- and model-dependent, while still allowing for proper gradient routing. For example, Figure 2 shows how easy it is to implement the "pattern-preserving[6] ablation" of prior work (Olsson et al., 2022). Importantly, our framework allows implementing such causal manipulations *throughout training*, which enables us to isolate the *dynamics* of subcircuit formation (Section 4.2). While prior work[7] was restricted to conducting mechanistic analyses on checkpoints from training, providing only correlational evidence, our framework allows *causal* interventions on learning dy-

[4] In Section 5, we vary these parameters to more deeply study the dependence of induction head formation on data properties.

[5] See Appendix B for details.

[6] Here "patterns" are used to refer to the post-softmax attention scores from one token to others.

[7] We note that some prior work in vision (Ranadive et al., 2023) or masked language models (Chen et al., 2024) has attempted to causally manipulate learning, but primarily through the use of regularizing losses. Our framework permits more direct interventions (on activations) throughout training.